Beyond Binary -- Trees for the Real World of Disks

Lecture 13

Data Structures and Algorithms

Part 1: Why Multi-Way Trees?

The Story So Far

We have spent several lectures studying binary trees -- BSTs, AVL trees, Red-Black trees. They all share one structural rule: each node has **at most 2 children**.

For data that lives entirely in **RAM**, binary trees work beautifully. AVL and Red-Black trees guarantee O(log n) search, insert, and delete.

But what happens when the data does not fit in memory?

Databases routinely store hundreds of millions of records. Search engines index billions of web pages. File systems manage millions of files. This data lives on **disk**, and disk access is governed by a completely different set of rules than memory access.

Today we leave the world of binary trees and enter the world of **multi-way trees** -- trees where each node can hold many keys and point to many children. This is where theory meets the machines that actually store our data.

The Disk Access Problem

Every data structure we have studied so far assumes that accessing any element takes the same amount of time. In RAM, this is roughly true -- reading from any memory address takes a few nanoseconds.

Disk is a completely different story.

Storage	Access Time	Relative Speed
CPU Cache (L1)	~1 ns	1x
RAM	~100 ns	100x slower
SSD	~100,000 ns	100,000x slower
Hard Disk (HDD)	~10,000,000 ns	10,000,000x slower

A single disk read is **100,000 to 10,000,000 times slower** than a RAM access. The difference is so dramatic that **the number of disk accesses dominates everything else**. We could do a million comparisons in RAM in the time it takes to read one block from a hard drive.

Why Binary Trees Fail on Disk

An AVL tree with **1 million nodes** has a height of about 20. That means a search requires visiting up to 20 nodes.

If each node is stored on disk, that is **20 disk accesses** per search. At 10 ms per access, a single lookup takes **200 milliseconds** -- nearly a quarter of a second. A database serving thousands of queries per second cannot afford that.

The root cause: Binary trees are tall and narrow. Each node holds 1 key and 2 pointers. Every "decision" at a node eliminates only half the data. With 1 million records, we need about 20 decisions, and each decision costs a full disk read.

The solution: Make each node hold **many keys** and **many children**. Instead of making one decision per disk read (left or right?), we make many decisions per disk read. This makes the tree **short and wide**.

Think of it this way: a library that puts 2 books on each shelf (binary tree) needs 20 floors to hold a million books. A library that puts 500 books on each shelf (multi-way tree) needs only 3 or 4 floors.

How Disk Reads Actually Work

When you read from disk, the operating system does not fetch a single byte. It fetches an entire **block** (also called a page), typically **4 KB to 16 KB** in size.

This means:

- Reading 1 byte costs the same as reading 4,000 bytes -- the entire block is loaded
- If a tree node is small (say 20 bytes for a BST node), most of the block is wasted
- If we design a node to **fill an entire block**, we get hundreds of keys "for free"

The key insight: Since we pay the full cost of a disk access for each node we visit, we should make each node as large as a disk block and pack it full of keys. This way, each disk access gives us maximum information.

A B-Tree node of order 1000 holds up to 999 keys in a single disk block. One disk read lets us make up to 999 decisions about where the target key lies. Compare that with a BST node, where one disk read gives us exactly one decision.

Multi-Way Search Trees

A **multi-way search tree** (or **m-way search tree**) generalizes the BST to nodes with more than 2 children.

Definition: An m-way search tree has the following properties:

- Each node has at most **m children** and at most **m - 1 keys**
- Keys within each node are stored in **sorted order**: $k_1 < k_2 < \ldots < k_{m-1}$
- For each key k_i, all keys in the subtree between k_i and k_{i+1} fall in the range (k_i, k_{i+1})

```
Example: A 4-way search tree node with 3 keys

     [10 | 30 | 50]
    /    |     |    \
   /     |     |     \
 <10   10-30  30-50   >50
```

Each key acts as a "signpost" that directs the search to the correct child subtree. A BST is simply a 2-way search tree -- one key, two children.

The Problem with Unbalanced Multi-Way Trees

Just like BSTs, a plain multi-way search tree has no rules about balance. We could build a tree where every node has only one key and two children -- degenerating right back to a BST.

```
        [10]
          \
         [20]
           \
          [30]
            \
           [40]

An m-way search tree with no balancing rules can become
just as lopsided as a BST. Height = n, Search = O(n).
```

The problem is not just that the tree is tall. It is that we have wasted space: nodes designed to hold many keys are storing only one each, and the wide branching factor we hoped for is not being used.

We need a self-balancing multi-way search tree -- one that enforces minimum and maximum occupancy for each node and keeps all leaves at the same level.

That is exactly what a **B-Tree** is.

Part 2: B-Tree Properties and Search

What is a B-Tree?

The B-Tree was invented by **Rudolf Bayer and Edward McCreight** in 1970 while working at Boeing Scientific Research Labs. It is a self-balancing multi-way search tree specifically designed for systems that read and write large blocks of data -- primarily **disk-based storage**.

Where B-Trees are used today:

- **MySQL** (InnoDB engine) -- every table index is a B+ Tree
- **PostgreSQL** -- default index type is a B-Tree
- **SQLite** -- the entire database is organized as B-Trees
- **File Systems:** NTFS (Windows), HFS+ (macOS), ext4 (Linux)
- **Key-value stores:** LevelDB, RocksDB, WiredTiger (MongoDB)

When you run a database query with a `WHERE` clause on an indexed column, the database is almost certainly walking down a B-Tree to find your data. When you open a folder on your computer, the file system is searching a B-Tree to locate the directory entries.

B-Trees are one of the most important data structures in all of computer science.

B-Tree of Order m -- The Rules

A B-Tree of order **m** (also called a B-Tree of degree m) must satisfy all of the following properties:

1. Every node has at most **m children** (and therefore at most **m - 1 keys**)
2. Every non-leaf, non-root node has at least $\lceil m/2 \rceil$ children (at least $\lceil m/2 \rceil - 1$ keys)
3. The root has at least **2 children** (unless it is a leaf -- i.e., the tree has very few keys)
4. **All leaf nodes appear at the same level** -- the tree is perfectly balanced
5. Keys within each node are stored in **sorted ascending order**

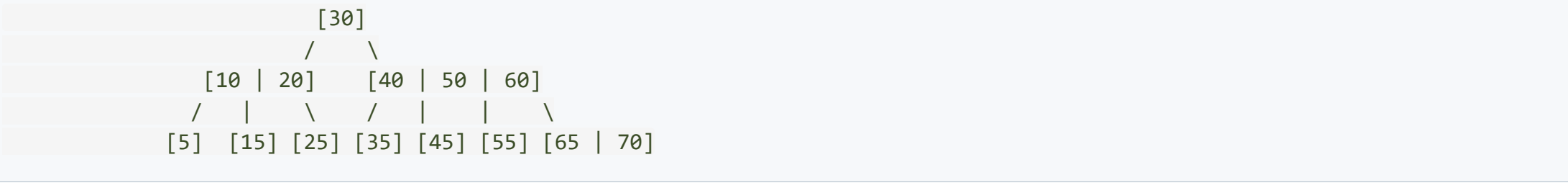
What do these rules accomplish?

- Rule 1 limits the maximum size (no node becomes too large)
- Rule 2 guarantees minimum occupancy (no node is too empty -- at least half full)
- Rule 3 gives the root special status (it can have fewer keys since it has no siblings)
- Rule 4 is the balance guarantee -- **all paths from root to any leaf have the same length**
- Rule 5 enables efficient search within a node (binary search or linear scan)

B-Tree of Order 5 -- A Concrete Example

For a B-Tree of order 5:

- Each node holds **1 to 4 keys** (maximum m-1 = 4)
- Each non-root internal node has **3 to 5 children** (minimum ceil(5/2) = 3)
- All leaves are at the same level



Verifying the properties:

Property	Check
Max 4 keys per node	Yes -- largest node has 2 keys (root has 1)
Non-root internal: at least 2 keys	Yes -- [10,20] has 2 keys, [40,50,60] has 3
Root has at least 2 children	Yes -- root [30] has 2 children
All leaves at the same level	Yes -- all at level 2
Keys sorted within each node	Yes -- check every node

Why B-Trees are So Efficient

The height of a B-Tree of order m with n keys is:

$$h = O\left(\log_{\lceil m/2 \rceil} n\right)$$

The base of the logarithm is $\lceil m/2 \rceil$, not 2. This makes a massive difference.

Real-world comparison

Suppose we have **1 billion keys** ($n = 10^9$):

Tree Type	Branching Factor	Height	Disk Accesses
BST (balanced)	2	~30	~30
B-Tree (order 100)	50	~6	~6
B-Tree (order 1000)	500	~4	~4
B-Tree (order 10000)	5000	~3	~3

A B-Tree of order 10,000 can search through **one billion records in just 3 disk accesses**. The root node is usually cached in RAM, so in practice, it takes only **2 disk reads** to find any record.

This is the reason databases can answer queries on tables with billions of rows in milliseconds.

B-Tree Search Algorithm

Searching a B-Tree is a natural generalization of BST search. Instead of comparing against one key and going left or right, we compare against **multiple keys** and choose the correct child.

Algorithm:

```
Search(node, key):
    1. Find the smallest index i such that key <= node.keys[i]
       (use binary search within the node for efficiency)

    2. If key == node.keys[i]:
       -- Found it! Return this node and index i.

    3. If node is a LEAF:
       -- Key is not in the tree. Return "not found".

    4. Otherwise:
       -- Read child[i] from disk
       -- Recursively search in child[i]
```

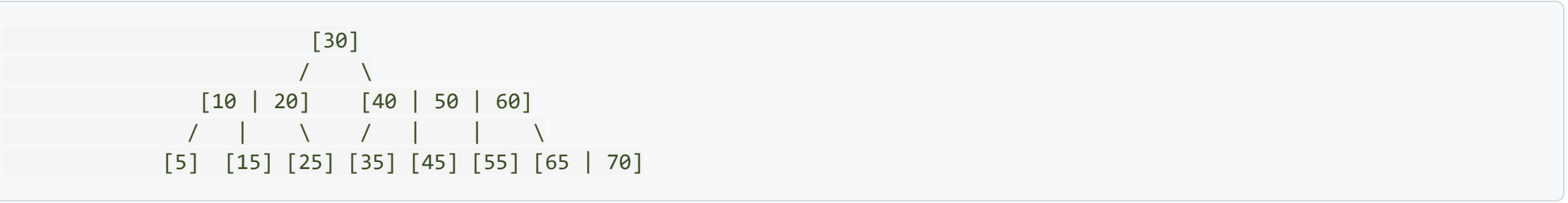
At each level, we do:

- One **disk read** (load the node into memory)
- One **in-memory search** among the node's keys (binary search: O(log m))

Total: $O(\log_m n)$ disk accesses, each with an $O(\log m)$ in-memory search.

B-Tree Search: Traced Example

Search for key **45** in this B-Tree of order 5:



Step 1: Start at root [30].

- Compare 45 with 30. Since 45 > 30, follow the **right** child pointer.
- **Disk access 1.**

Step 2: Arrive at node [40 | 50 | 60].

- Compare 45 with 40. Since 45 > 40, move to next key.
- Compare 45 with 50. Since 45 < 50, follow the child pointer **between 40 and 50**.
- **Disk access 2.**

Step 3: Arrive at leaf [45].

- Compare 45 with 45. **Match found.**
- **Disk access 3.**

Total: 3 disk accesses. The in-memory comparisons within each node are essentially free.

Part 3: B-Tree Insertion

B-Tree Insertion Strategy

Insertion in a B-Tree follows a fundamentally different approach from BSTs. In a BST, new nodes are always added as leaves at the bottom. B-Trees also insert into leaf nodes, but the interesting part is what happens when a leaf is already full.

The algorithm:

1. **Search** for the correct leaf node where the new key belongs
2. **If the leaf has room** (fewer than m-1 keys): insert the key in sorted position. Done.
3. **If the leaf is full** (already has m-1 keys): the node **overflows** and must be **split**

The splitting process is the heart of B-Tree insertion. It is what keeps the tree balanced while allowing it to grow.

Unlike BSTs where the tree grows downward (new leaves are added at the bottom), **B-Trees grow upward**. When a split cascades all the way to the root, a new root is created, and the height of the entire tree increases by one. This is the only way a B-Tree becomes taller.

The Splitting Process

When a node has **m keys** after an insertion (one more than the maximum allowed), it overflows and is split:

1. Find the **median key** (the middle element)
2. **Promote** the median key up to the parent node
3. Split the remaining keys into two halves:
   - Keys **less than** the median stay in the **left** node
   - Keys **greater than** the median go into a new **right** node
4. Update the parent's child pointers to include the new node

```
Before (order 5, node overflows with 5 keys):

    [10 | 20 | 30 | 40 | 50]      <-- overflow! max is 4 keys

After splitting at median (30):

              [30]                 <-- median promoted to parent
             /    \
    [10 | 20]      [40 | 50]      <-- split into two nodes
```

If the parent also overflows after receiving the promoted key, it too is split. This can cascade upward. If the **root** splits, a new root is created, and the tree height increases by 1.

Insertion Example: B-Tree of Order 5

Let us build a B-Tree of order 5 by inserting: **10, 20, 30, 40, 50, 60, 70, 80**

Each node can hold at most **4 keys**.

Insert 10, 20, 30, 40:

All four keys fit in a single root node. No splitting needed.

```
    [10 | 20 | 30 | 40]
```

Insert 50:

The root now has 5 keys -- overflow! Split at the median (30):

```
Before:   [10 | 20 | 30 | 40 | 50]     <-- overflow!

After:             [30]                   <-- 30 promoted to new root
                  /    \
        [10 | 20]      [40 | 50]

Height increased from 0 to 1. The tree grew upward.
```

Insertion Example (continued)

Insert 60:

60 > 30, so it goes into the right child [40 | 50]. Room available.

```
            [30]
           /    \
  [10 | 20]     [40 | 50 | 60]
```

Insert 70:

70 goes into the right child [40 | 50 | 60]. Room available (max 4 keys).

```
            [30]
           /    \
  [10 | 20]     [40 | 50 | 60 | 70]
```

Insertion Example (continued)

Insert 80:

80 goes into the right child, which already has 4 keys. Adding 80 causes **overflow**:

```
Before:
             [30]
            /    \
   [10 | 20]     [40 | 50 | 60 | 70 | 80]     <-- overflow!

Split right child at median (60), promote 60 to parent:

             [30 | 60]
            /    |    \
   [10 | 20]  [40 | 50]  [70 | 80]
```

The root absorbed the promoted key 60, and the right child split into two nodes. The root now has 2 keys and 3 children -- perfectly valid.

Notice the pattern: The tree grows wider at the bottom and taller at the top. Splits push keys upward, not downward. This is why all leaves always remain at the same level.

Insertion Example: Adding 5 and 15

Continuing from the previous tree, let us insert **5** and **15**.

Insert 5:

5 < 30, so it goes into the left child [10 | 20]. Room available.

```
              [30 | 60]
             /    |    \
  [5 | 10 | 20]  [40 | 50]  [70 | 80]
```

Insert 15:

15 < 30, so it goes into the left child [5 | 10 | 20]. Room available (max 4 keys).

```
                [30 | 60]
               /    |    \
  [5 | 10 | 15 | 20]  [40 | 50]  [70 | 80]
```

The left child is now full (4 keys). The next insertion into this child will cause a split.

When Splits Cascade to the Root

Consider a scenario where the root itself overflows. Suppose after several more insertions, the root [30 | 60] has absorbed two more promoted keys and becomes:

```
              [20 | 30 | 50 | 60]              <-- root is full (4 keys)
             / |    |    |     \
           ... ...  ...  ...   ...

Insert a value that causes a child to split, promoting a key into the root:

          [20 | 30 | 40 | 50 | 60]              <-- root overflows!

Split root at median (40):

                      [40]                        <-- new root created!
                     /    \
           [20 | 30]       [50 | 60]
          /  |    \        /   |    \
        ... ... ...      ... ...  ...
```

The tree height increases by 1. This is the **only** way a B-Tree gets taller -- when the root splits, a new root with a single key is created above it. Every leaf is still at the same level.

Insertion Complexity

Component	Cost
Finding the correct leaf	O(log m) comparisons at each of O(h) levels
Splitting a node	O(m) work to redistribute keys
Cascading splits (worst case)	One split per level = O(h) splits
Total disk accesses	**O(h)** = $O(\log_{\lceil m/2 \rceil} n)$

In practice, splits are rare. Most insertions simply add a key to a non-full leaf -- a single disk read and a single disk write. The tree is designed so that nodes are at least half full, which means roughly half the capacity is available for new keys at any time.

For a B-Tree of order 1000 with 1 billion keys:

- Height is about 3
- An insertion requires at most 3 disk reads + 3 disk writes
- Root is typically cached, so in practice: **2 reads + a write**

Part 4: B-Tree Deletion

Why Deletion is Complex

Deletion in a B-Tree must maintain all five properties, most critically:

- Every non-root node has at least $\lceil m/2 \rceil - 1$ keys (**minimum occupancy**)
- All leaves are at the same level

When we remove a key from a node, the node may drop below the minimum key count. This is called an **underflow**. Handling underflow is what makes B-Tree deletion complex.

Two main cases:

Case	Description
Delete from a leaf	Straightforward if the leaf has spare keys. If not, we must borrow or merge.
Delete from an internal node	Replace the key with its in-order predecessor or successor (always in a leaf), then delete from the leaf.

The second case always reduces to the first. So the core challenge is handling leaf underflow.

Case 1A: Delete from a Leaf -- Node has Spare Keys

This is the simplest case. The leaf has **more than the minimum** number of keys. Simply remove the key, and we are done.

Example: Delete **70** from this B-Tree of order 5 (min 2 keys per non-root node):

```
Before:
                [30 | 60]
               /    |    \
   [5 | 10 | 20]  [40 | 50]  [65 | 70 | 80]

After deleting 70:
                [30 | 60]
               /    |    \
   [5 | 10 | 20]  [40 | 50]  [65 | 80]
```

The right leaf had 3 keys, which is above the minimum of 2. Removing one key leaves it with 2, which is still valid. No restructuring needed.

Case 1B: Delete from a Leaf -- Borrow from a Sibling

The leaf has exactly the minimum number of keys. Removing one would cause underflow. But an **adjacent sibling has spare keys**.

Solution: Borrow a key from the sibling through the parent -- a **rotation through the parent**.

Example: Delete **25** from this B-Tree of order 5:

```
Before:
                    [30]
                   /    \
          [20 | 25]   [40 | 50 | 60]

After (borrow from right sibling through parent):
                    [40]
                   /    \
            [20 | 30]  [50 | 60]
```

What happened:

1. Key 25 is removed from the left child -- it would underflow (only 1 key left, minimum is 2)
2. The parent's separator key (30) moves **down** into the left child
3. The right sibling's **smallest key** (40) moves **up** to replace the separator in the parent
4. Both children now have valid key counts

Case 1C: Delete from a Leaf -- Merge with a Sibling

The leaf has the minimum number of keys, and **no adjacent sibling has spare keys**. We cannot borrow.

Solution: Merge the leaf with an adjacent sibling and pull the separating key **down** from the parent.

Example: Delete **25** from this B-Tree of order 5:

```
Before:
                    [30]
                   /    \
            [10 | 25]    [40 | 50]

After deleting 25 (merge left and right children with separator):

            [10 | 30 | 40 | 50]
```

What happened:

1. Key 25 is removed from the left child, leaving [10] -- below minimum (need 2)
2. The right sibling [40 | 50] has exactly the minimum -- cannot lend a key
3. Merge: combine the left child, the parent's separator key (30), and the right sibling
4. The parent gave away its only key, so it becomes empty
5. The merged node becomes the new root

The tree height decreased by 1. This is the **only** way a B-Tree shrinks.

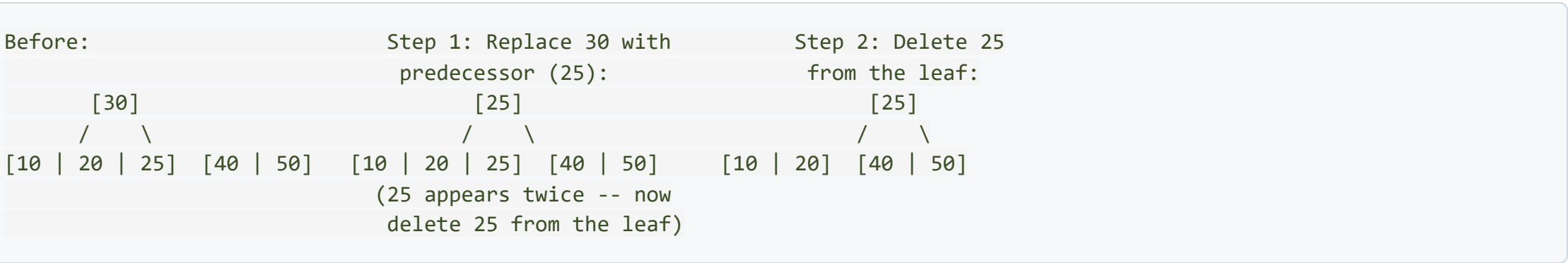
Case 2: Delete from an Internal Node

If the key to be deleted is in an internal node (not a leaf), we **replace** it with one of two candidates:

- **In-order predecessor:** The largest key in the left child subtree (rightmost key of the rightmost leaf in the left subtree)
- **In-order successor:** The smallest key in the right child subtree (leftmost key of the leftmost leaf in the right subtree)

Either one is always located in a **leaf node**. After replacing, we delete the replacement key from the leaf, which reduces to Case 1.

Example: Delete **30** from an internal node:



Cascading Merges

When a merge occurs, the parent loses a key. If the parent was at minimum occupancy, it too underflows. This triggers another borrow-or-merge at the parent level. The process can cascade all the way to the root.

```
Before (order 3 -- each node has 1-2 keys, 2-3 children):

                    [20]
                   /    \
              [10]        [30]
             /    \      /    \
          [5]  [15]  [25]   [35]

Delete 5:
  - Leaf [5] underflows
  - Sibling [15] has minimum keys -- cannot borrow
  - Merge [5] and [15] with separator 10:

                    [20]
                   /    \
             [10 | 15]    [30]
                         /    \
                      [25]   [35]

  Wait -- actually [5] is gone, so the merged node is [10 | 15]. No cascade needed here.
```

If a merge at one level causes the parent to also underflow, the parent must also borrow or merge. If the root ends up with zero keys, it is removed, and its single child becomes the new root.

Deletion Complexity

Component	Cost
Finding the key	O(h) disk accesses with O(log m) comparisons each
Replacing with predecessor/successor	O(h) at worst
Borrow or merge at each level	O(m) work per level
Cascading merges (worst case)	One per level = O(h)
Total disk accesses	**O(h)** = $O(\log_{\lceil m/2 \rceil} n)$

Just like insertion, the worst case involves restructuring at every level. But in practice, most deletions either remove a key from a non-minimal leaf (no restructuring) or borrow from a sibling (local operation). Cascading merges are rare.

All three B-Tree operations -- search, insert, delete -- have the same complexity: O(h) disk accesses. The height h is logarithmic with a large base, making B-Trees extraordinarily efficient for disk-based storage.

B-Tree Operations -- Complete Summary

Operation	Disk Accesses	In-Memory Work
Search	O(h)	O(log m) per node
Insert	O(h)	O(m) for splitting
Delete	O(h)	O(m) for merging

Where $h = O(\log_{\lceil m/2 \rceil} n)$.

For practical systems (m = 200, n = 100 million):

Operation	Disk Accesses
Search	3-4
Insert	3-4 (usually no split)
Delete	3-4 (usually no merge)

Each disk access reads or writes a full block (4-16 KB), which holds one node. The in-memory work within each node is essentially instantaneous compared to the disk access time.

Part 5: B-Tree Variants and Applications

B+ Trees -- The Database Favorite

Databases almost universally use a variant called the **B+ Tree** instead of a plain B-Tree. The key difference:

Feature	B-Tree	B+ Tree
Data storage	In **all** nodes	Only in **leaf** nodes
Internal nodes contain	Keys + data + child pointers	Keys + child pointers only
Leaf nodes	Independent	**Linked together** in a chain
Range queries	Must traverse the tree	Follow the leaf chain

```
B+ Tree structure:

          [30 | 60]                      <-- internal: keys only, no data
         /    |    \
  [10|20]  [30|40|50]  [60|70|80]       <-- leaves: keys + data
    |---->   |-------->   |              <-- leaves linked in order
```

Why databases prefer B+ Trees:

- Internal nodes hold **more keys** (no data = more room for keys = shorter tree)
- **Range queries** (e.g., `WHERE age BETWEEN 20 AND 30`) just follow the leaf chain
- **Sequential scans** are efficient -- leaves are a linked list in sorted order
- All lookups end at a leaf -- **predictable performance**

B* Trees -- Delayed Splitting

Another variant is the *B Tree**, which improves space utilization. In a regular B-Tree, nodes are guaranteed to be at least **half full**. B* Trees guarantee nodes are at least **two-thirds full**.

How? Instead of splitting immediately when a node overflows, a B* Tree first tries to **redistribute keys** to an adjacent sibling. Only if both the node and its sibling are full does a split occur -- and it splits **two nodes into three**.

Property	B-Tree	B* Tree
Minimum fill	50%	67%
Split strategy	1 node becomes 2	2 nodes become 3
Space utilization	Good	Better
Complexity	Moderate	Higher

B* Trees are less common than B+ Trees, but they demonstrate an important principle: there are always trade-offs between space utilization, implementation complexity, and performance.

Applications: Database Indexing -- The Killer App

When you create an index on a database table, you are building a B-Tree (or B+ Tree):

```
SQL:   CREATE INDEX idx_age ON students(age);

What the database does internally:

    Build a B+ Tree where:
      - Keys = age values from every row
      - Leaf data = pointers to the actual rows on disk

    Before index: SELECT * FROM students WHERE age = 21;
        --> Full table scan: reads ALL rows (could be millions)

    After index:  SELECT * FROM students WHERE age = 21;
        --> B+ Tree search: 3-4 disk reads to find the rows
```

Table Size	Without Index	With B-Tree Index
1,000 rows	up to 1,000 reads	2-3 reads
1,000,000 rows	up to 1,000,000 reads	3-4 reads
1,000,000,000 rows	up to 1 billion reads	3-4 reads

The difference between minutes and milliseconds.

Applications: File Systems

When you navigate to a folder on your computer and list files, the operating system is searching a B-Tree.

NTFS (Windows):

- The Master File Table (MFT) is organized as a B+ Tree
- File attributes, directory entries, and security descriptors are indexed using B-Trees
- Each folder's contents are stored as a B-Tree sorted by filename

ext4 (Linux):

- Uses a structure called an HTree (essentially a B-Tree variant) for directory indexing
- Without it, listing files in a directory with 100,000 entries would require reading every single entry

HFS+ (macOS):

- The entire volume catalog (all files and folders) is a single B-Tree
- File metadata is stored in another B-Tree

In every case, the principle is the same: B-Trees minimize the number of disk accesses needed to find information, which is the bottleneck for any storage system.

"When Do I Use What?" -- The Complete Guide

Scenario	Best Choice	Why
Fast search in memory	**AVL Tree**	Strictly balanced, shortest tree
Frequent insert and delete in memory	**Red-Black Tree**	Bounded rotations, cheaper maintenance
Data stored on disk	**B-Tree / B+ Tree**	Minimizes expensive disk accesses
Priority queue, sorting	**Binary Heap**	O(1) find-max, O(log n) insert
Database indexing	**B+ Tree**	Range queries via linked leaves
File system directories	**B-Tree**	Fast lookup of filenames on disk
Compiler symbol table	**AVL Tree**	Built once, searched many times
Language standard library (general)	**Red-Black Tree**	Good all-around insert/delete/search

The right data structure depends on two things: **where the data lives** (memory vs. disk) and **what operations dominate** (search vs. insert/delete vs. range queries).

Key Takeaways

What We Covered Today

1. The Disk Access Problem

- Disk is 100,000x to 10,000,000x slower than RAM
- The number of disk accesses is the dominant cost for disk-based data structures
- Binary trees are too tall and narrow for disk -- each node wastes a full block read

2. Multi-Way Search Trees and B-Trees

- Multi-way trees generalize BSTs: each node holds multiple keys and has multiple children
- B-Trees enforce balance: all leaves at the same level, nodes at least half full
- Height is $O(\log_{\lceil m/2 \rceil} n)$ -- with order 1000, a billion keys need only 3 levels

3. B-Tree Operations

- Search: navigate down the tree using keys as signposts, one disk access per level
- Insert: always into a leaf; split and promote when full; tree grows upward
- Delete: borrow from sibling or merge; cascading merges can shrink the tree

4. B-Tree Variants and Applications

- B+ Trees: data only at leaves, leaves linked for range queries -- used by every major database
- B-Trees power database indexing, file systems, and key-value stores

Review Questions

1. A B-Tree of order 7 has how many minimum and maximum keys per non-root node? How many minimum and maximum children?
2. Why does a B-Tree grow **upward** (by splitting the root) rather than downward (by adding new leaves)? What property of the B-Tree does this preserve?
3. Insert the keys **5, 8, 1, 7, 3, 12, 9, 6** into an initially empty B-Tree of order 3. Show the tree after each split.
4. Suppose you have a B-Tree of order 1001 containing 1 billion keys. What is the maximum height of the tree? How many disk accesses are needed for a search?
5. Explain the difference between a B-Tree and a B+ Tree. Why do databases prefer B+ Trees for indexing?
6. Delete the key **40** from the B-Tree shown in slide 16 (the final tree after inserting 10-80). Show each step.
7. A database table with 50 million rows has a B+ Tree index of order 200. Approximately how many disk reads are needed to find a single row? What about a full table scan without the index?
8. Why is it said that B-Trees are "optimized for disk" while AVL trees are "optimized for memory"? What fundamental assumption differs?

Next Lecture

Binary Heaps and Heap Sort -- The Grand Finale

- Binary Heaps: complete binary trees stored in arrays, no pointers needed
- Heap operations: insert (swim up) and delete-max (sink down)
- Build-Heap in O(n) using Floyd's bottom-up algorithm
- Heap Sort: O(n log n), in-place, and guaranteed -- the unit's final sorting algorithm